Teaching a KAN to Copy a Bent Line

A plain-language guide to Nadia's example for the Operations Research Forum revision

Who this is for. Anyone who wants to understand what the example does and what it shows, without assuming any background in mathematics or machine learning. Technical words are explained the first time they appear and collected in a glossary at the end. Every claim is backed by a number in Section 7.

Contents

1. The question the example answers
2. Two bent lines: what S1 and S2 are
3. How Nadia's code builds them, step by step
4. What a KAN is, and what its 'grid' means
5. How a KAN is trained
6. What we tested
7. The results
8. What the results mean for the paper
9. Glossary

1. The question the example answers

The paper compares two kinds of learning model, called KAN and LSTM, on stock prices. One reviewer, a mathematician who works on approximation, raised a point about how KANs are built on the inside. Nadia's answer is a small, clean example that has nothing to do with stock prices. It asks a very simple question:

> **If you know exactly what a function looks like, and it is as simple as a few straight lines joined together, can a small KAN copy it? And does it matter *where* the lines join?**

Why ask this? Because the answer exposes something about how a KAN works that is invisible on noisy stock data: a KAN can only bend its curves at certain fixed positions. If the thing it is copying bends somewhere else, it struggles. The example makes that concrete with two functions that are identical in every respect except where they bend.

A 'function' here just means a rule that turns one number into another. Give it a value of *t* between 0 and 10, it hands back a value *S(t)*. Draw all those pairs and you get a line on a graph. The two functions are called S1 and S2.

2. Two bent lines: what S1 and S2 are

Both S1 and S2 are **piecewise linear**: made of straight pieces joined end to end, like a bent wire. Each place where the wire bends is called a **knot** (mathematicians' word) or a **kink** (the everyday word). Between knots the line is perfectly straight; at a knot it changes direction.

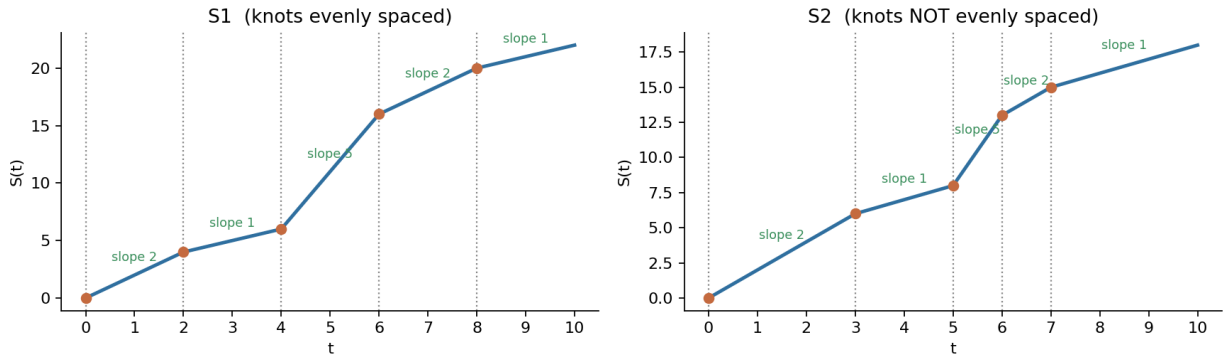


Figure 1. S1 (left) and S2 (right). Orange dots are the knots, the places where each line bends. The green labels show the slope of each straight piece, how steeply it rises or falls. The slopes are the same five numbers in both pictures, in the same order. The only difference is where the bends happen: S1 bends at 0, 2, 4, 6, 8 (evenly spaced), S2 at 0, 3, 5, 6, 7 (bunched up in the middle).

Look at the two pictures and notice: the same five steepnesses, in the same order. First a gentle rise, then gentler, then steep, then gentle, then almost flat. S2 is simply S1 with the bends shifted. This is deliberate. Any difference in how well a KAN copies them cannot be blamed on one function being 'harder' in the usual sense. The only variable is knot placement.

Each function is given to the computer as a list of 1,001 points: the value of *S(t)* at *t* = 0, 0.01, 0.02, and so on up to 10. That is what the files S1.txt and S2.txt contain.

3. How Nadia's code builds them, step by step

Nadia's Matlab script is short. Here it is in full, and then what each part does.

```
A = [2 -1 4 -3 -1]; % five slopes
Theta1 = [0, 2, 4, 6, 8]; % knots of S1 (evenly spaced)
```

```
Theta2 = [0, 3, 5, 6, 7]; % knots of S2 (not evenly spaced)
for each t from 0 to 10 in steps of 0.01:
    S1(t) = sum over j of A(j) * max(0, t - Theta1(j))
    S2(t) = sum over j of A(j) * max(0, t - Theta2(j))
```

The building block: a hinge. Everything rests on one tiny piece: **max(0, t - 3)**. Read it as 'take *t* minus 3, but if that would be negative, use 0 instead'. Before *t* reaches 3 the result is 0, flat. After 3 it rises in a straight line. The shape is a hinge, or a hockey stick, and the point where it starts rising is its knot. Multiply it by a number and the stick gets steeper, or flips to point downward.

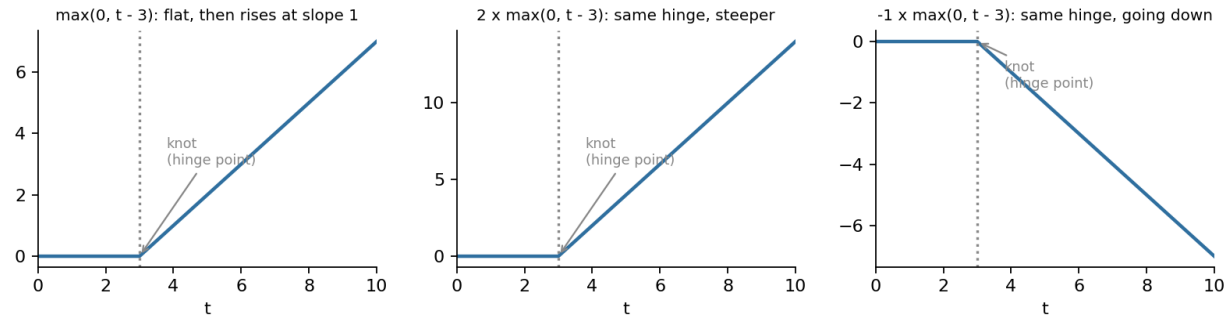


Figure 2. The hinge max(0, t - 3) and two multiples of it. Flat, then straight; the knot is where it changes.

Adding hinges. Now add five hinges together, each starting at a different knot and each multiplied by one of the five slopes. Every time *t* passes a new knot, one more hinge 'switches on' and the overall steepness changes by that hinge's slope. The result is a bent line with a bend at every knot. Figure 3 shows S1 being assembled one hinge at a time.

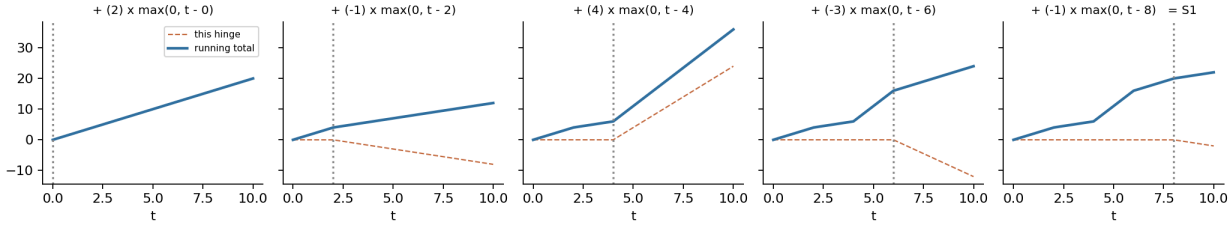


Figure 3. Building S1 by adding one hinge at a time (dashed orange = the hinge being added, blue = the running total). After five hinges, the total is S1. S2 is built the same way with the knots moved.

That is the whole of the script. The **for** loop just walks through the 1,001 values of *t*, and for each one adds up the five hinges. The last two lines save the results to the text files. There is nothing hidden: the functions are exactly the sum of five hinges, which is why we know precisely where every bend is.

4. What a KAN is, and what its 'grid' means

A **KAN** (Kolmogorov-Arnold Network) is a learning model. Most such models have fixed shapes inside and learn only numbers. A KAN is the other way round: the thing it learns is the *shape of a curve*. The smallest KAN, written [1,1], has one input, one output, and a single learned curve connecting them (Figure 4, left). Give it *t*, it looks up its curve at *t*, and hands back the height. Training a [1,1] KAN on S1 means: find the curve shape that best matches S1. If it can take exactly the shape of S1, the copy will be perfect.

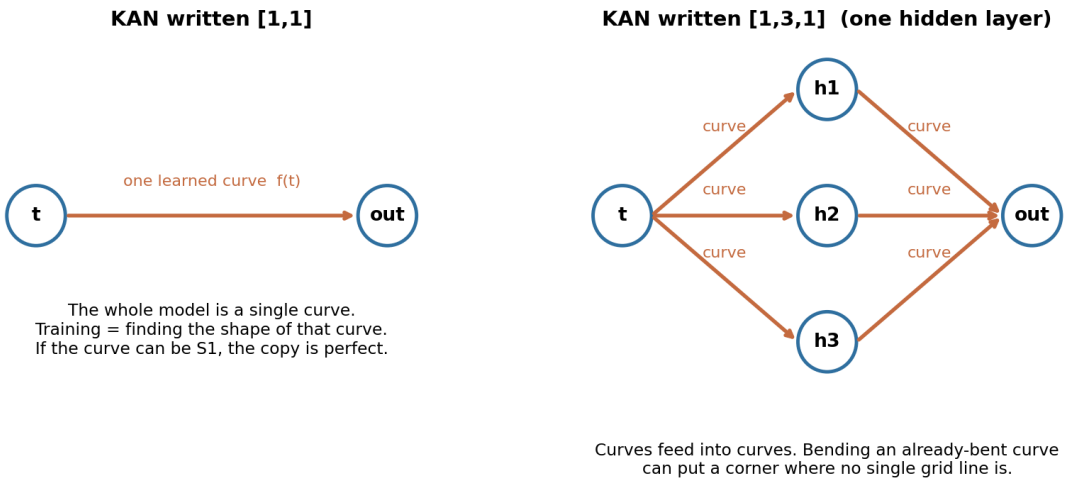


Figure 4. Left: the smallest KAN, one learned curve. Right: a KAN with a hidden layer of three units; the output is a curve of a sum of curves of the input.

How the curve is stored: the grid. A computer cannot store 'any curve'. A KAN stores its curve as a **spline**: a line made of small pieces, with the join points fixed in advance at evenly spaced positions. Those fixed positions are the **grid**. A '5-interval grid' on 0 to 10 puts join points at 0, 2, 4, 6, 8, 10. The KAN is free to choose how high the curve is at each grid point, but it is *not* free to move the grid points. So a KAN's curve can bend at grid points and nowhere else.

This is the whole crux of the example. Compare where S1 and S2 bend with where a 5-interval grid allows bending:

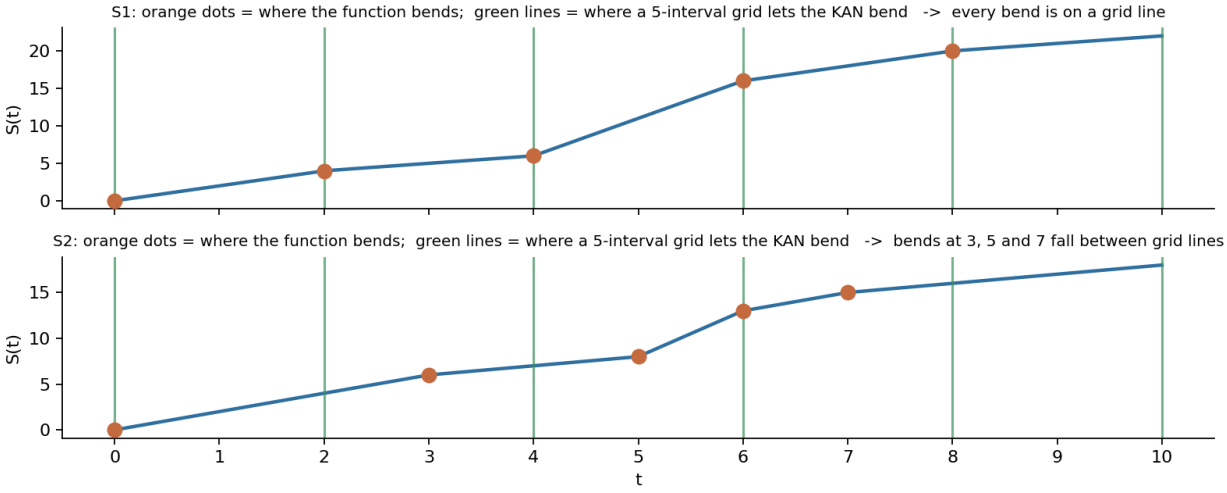


Figure 5. Green vertical lines: where a 5-interval grid lets the KAN bend. Orange dots: where the function actually bends. For S1 every bend sits on a green line. For S2, the bends at 3, 5 and 7 fall between them, where the KAN cannot bend.

The simplest spline: straight pieces. With 'linear' splines (written $k = 1$), the curve is straight between grid points. Technically it is a sum of little tent shapes, one centred on each grid point (Figure 6, left); choosing the height of each tent and adding them gives any straight-piece line with bends at the grid points. With the right six heights you get S1 exactly (Figure 6, right).

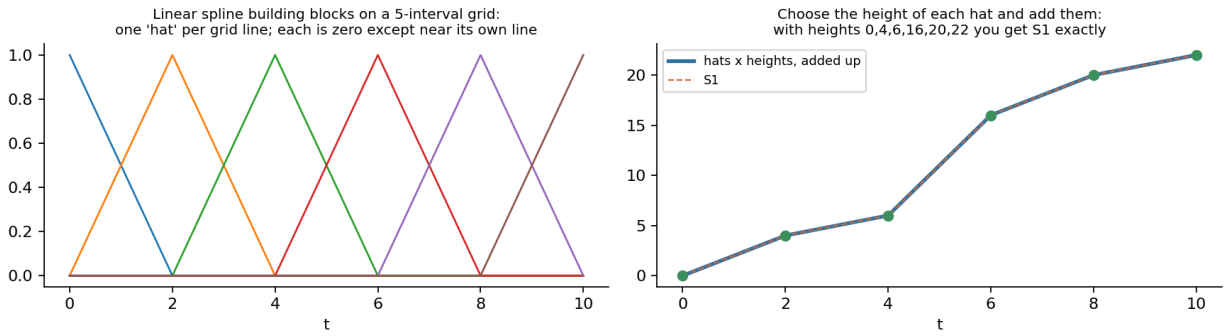


Figure 6. Left: the six tent-shaped building blocks of a linear spline on a 5-interval grid. Right: scale each tent and add them, and S1 appears exactly, because S1's bends are at the grid points.

Smooth splines. The paper's KANs use *cubic* splines ($k = 3$): the pieces are smooth curves rather than straight segments, and they join without any visible corner. That smoothness is normally a virtue, but a smooth curve *cannot* make a sharp corner at all. It can only round one off (Figure 7, left). So with cubic splines neither S1 nor S2 can ever be copied exactly, whatever the grid.

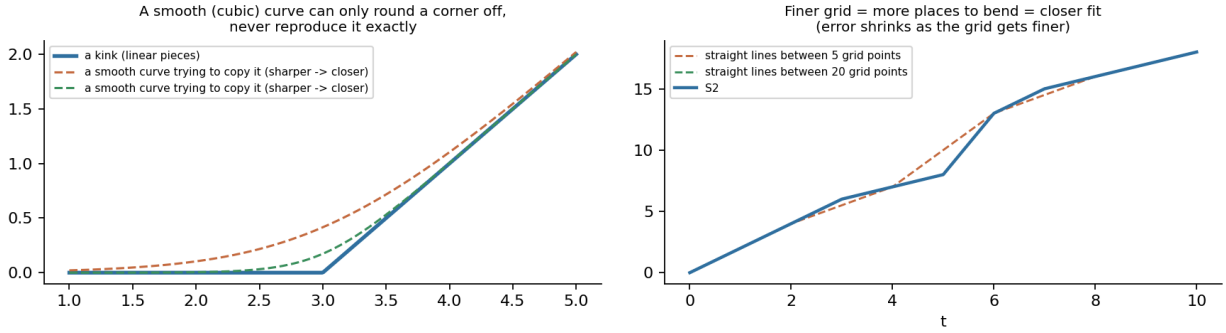


Figure 7. Left: a smooth curve can approach a corner but never reproduce it. Right: making the grid finer gives more places to bend, so the copy of S2 improves, at the cost of more numbers to learn.

5. How a KAN is trained

'Training' means finding the heights at the grid points (and a few other internal numbers) that make the KAN's curve match the target as closely as possible. The computer does not know the answer in advance; it searches for it.

The procedure is the same for every learning model:

1. Start with the heights set to small random values. The KAN's curve is nonsense.
2. Feed in all 1,001 values of t, and compare the KAN's output with the true $S(t)$ at each one. Add up how wrong it is. That total is called the **loss** or **error**.
3. Work out, for each height, which direction would reduce the error, and nudge every height a little that way.
4. Repeat steps 2 and 3. Each repeat is one **step**. We used 200 steps.

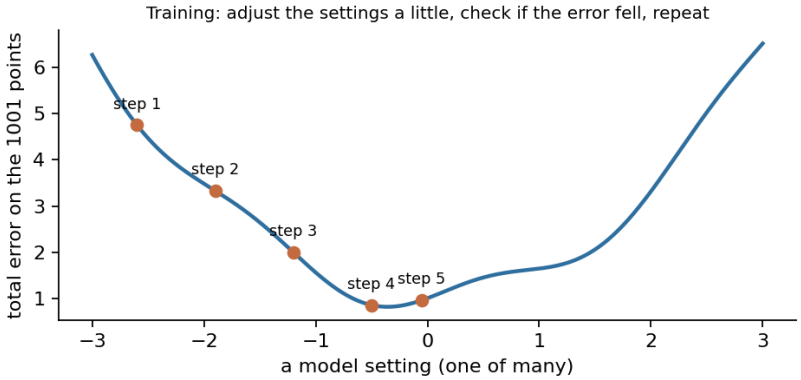


Figure 8. Training as walking downhill: each step adjusts the model's settings so the total error falls. The method used here, L-BFGS, takes big well-aimed steps and is standard for small problems like this.

Two details worth knowing. First, we told the KAN that *t* lives between 0 and 10, so its grid covers that range. Second, we did not let the KAN move its grid during training (a PyKAN option called 'grid updating'), because the point of the example is to see what a *fixed* grid can and cannot do.

How we measure success. After training, we compute the KAN's output at all 1,001 points and look at the single **worst** error anywhere, the largest gap between the KAN's line and the true line. Nadia's field, approximation theory, measures quality this way: a copy is only as good as its worst point. We also report the root-mean-square error (RMSE), the paper's usual measure, which is more like an average.

6. What we tested

Four experiments, each on both S1 and S2:

A. One curve, straight pieces, grid size varied. The smallest KAN ([1,1]) with linear splines, trying grids of 5, 10, 20, 50 and 100 intervals. This is the cleanest test of 'can it bend in the right places'.

B. One curve, smooth pieces, grid size varied. The same, but with cubic splines, as in the paper.

C. A hidden layer. KANs of shape [1,2,1] and [1,3,1] (Figure 4, right) on coarse grids, to see whether stacking curves can compensate for a grid that does not line up with the bends. This is the 'one or two layers, very few units' setting Nadia suggested.

D. A reference that is allowed to move its knots. Not a KAN: a plain bent line whose five knot positions *and* five slopes are both learned. If the search finds the right knots, this should copy both functions perfectly with only 11 numbers. It is the 'free-knot spline' the reviewer mentions, included to show what the KAN is being compared against.

7. The results

All numbers are the worst error over the 1,001 points. For scale, S1 rises to 22 and S2 to 18, so an error of 1.5 is about 7% of the function's height, clearly visible on a graph; an error of 0.02 would be invisible.

Experiment A: one curve, straight pieces

Grid intervals	Where the KAN can bend	S1 worst error	S2 worst error	Numbers learned
5	0, 2, 4, 6, 8, 10	0 (exact)	1.476	12
10	every whole number	0 (exact)	0 (exact)	17
20	every 0.5	0.024	0.020	27
50	every 0.2	0.004	0.003	57

100	every 0.1	0.001	0.001	107

The headline is the first row. With a 5-interval grid, the KAN copies S1 *perfectly* and misses S2 by 1.48. Same KAN, same number of settings, same training. The only difference is that S1's bends sit on the grid and S2's do not. Figure 9 shows what that 1.48 looks like: the KAN's line cuts across S2's corners because it is not allowed to bend at 3, 5 or 7.

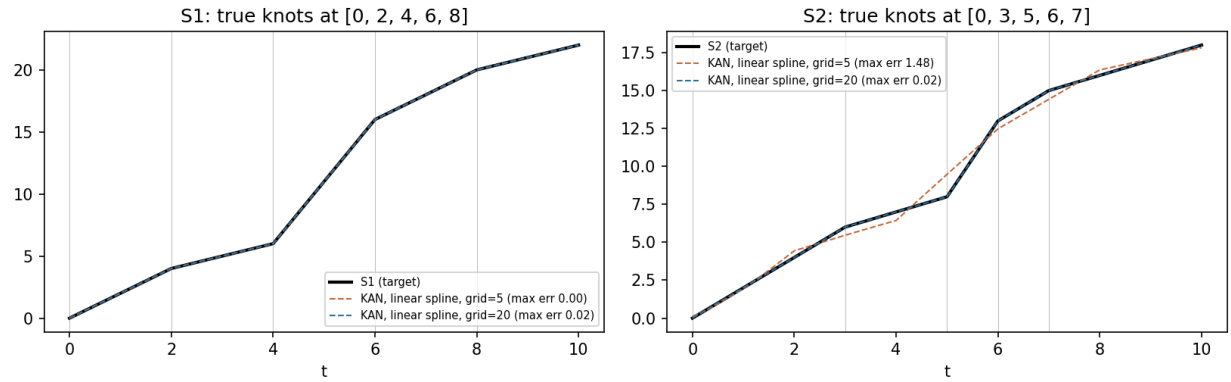


Figure 9. The actual fitted lines. Left, S1: a 5-interval KAN (orange dashed) sits exactly on top of the target. Right, S2: the same KAN cuts the corners; a 20-interval KAN (blue dashed) nearly fixes it.

Row two is a lucky coincidence worth understanding. With 10 intervals the grid lines are at every whole number, and S2's knots (3, 5, 6, 7) happen to be whole numbers, so they land on the grid and S2 becomes exact as well. Had Nadia chosen a knot at, say, 3.3, no uniform grid of any reasonable size would hit it. This is why the suggestion at the end of this document is to move S2's knots off the whole numbers.

Rows three to five show small errors creeping back in for both functions. That is not a representability problem (a finer grid can always bend where a coarser one could); it is that more grid points means more numbers to learn, and 200 training steps no longer finds the perfect answer. The error keeps falling with grid size all the same.

Experiment B: one curve, smooth pieces

Grid intervals	S1 worst error	S2 worst error
5	0.535	0.882
10	0.344	0.315
20	0.163	0.170
50	0.064	0.063
100	0.032	0.032

With smooth (cubic) splines neither function is ever exact, as predicted in Section 4: a smooth curve cannot make a corner. The error halves roughly every time the grid doubles, and crucially S1 and S2 now behave almost identically. Once the KAN cannot make corners anyway, it no longer matters much where the corners are. This is the setting the paper uses, so it is the more relevant row for the stock-price work.

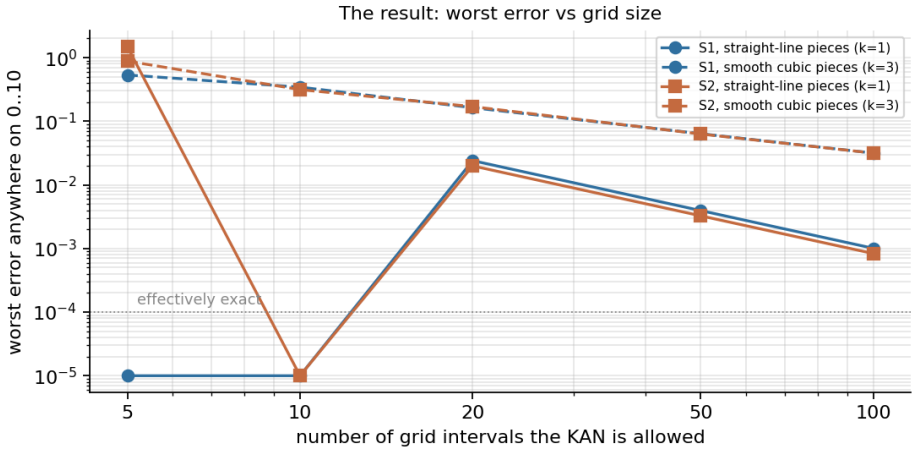


Figure 10. Experiments A and B on one chart (log scales). Solid lines: straight pieces; dashed: smooth pieces. The straight-piece S1 line sits at zero (exact) from the start; S2 joins it only once the grid reaches 10.

Experiment C: a hidden layer

KAN shape	Grid	S1 worst error	S2 worst error	Numbers learned
[1,2,1]	5	0.066	1.064	48
[1,2,1]	10	0.020	0.023	68
[1,3,1]	5	0.016	0.030	72
[1,3,1]	10	0.047	0.073	102

Stacking curves helps S2 a great deal on the coarse grid: [1,3,1] on 5 intervals brings the worst error from 1.48 down to 0.03. Feeding a bent curve into another bent curve can produce an effective bend at a position that is not on the grid. The price is six times as many numbers to learn, and the search gets harder: [1,3,1] on the finer 10-interval grid is actually *worse* than on 5, because training got stuck in a poor solution. More capacity is not automatically better.

Experiment D: the free-knot reference

Function	Worst error	Knots the search found	True knots
S1	0 (exact)	0, 2, 4, 6, 8	0, 2, 4, 6, 8
S2	0 (exact)	0, 3, 5, 6, 7	0, 3, 5, 6, 7

When the knots themselves are allowed to move, both functions are recovered exactly with just 11 numbers, and the search finds the true knot positions. But this came with a catch that is the reviewer's point in miniature: the search only succeeded for S2 when run from 14 different random starting positions and the best kept. With 6 starts it settled on wrong knots (3, 4.3, 5.0, 6.3) and an error of 0.48. Finding knot positions is a hard search with many false bottoms. Fixing the grid in advance, as KANs do, removes that difficulty, and the cost is exactly what Experiment A showed.

8. What the results mean for the paper

For the reviewer's question. The example demonstrates, in the cleanest possible setting, that a KAN's accuracy depends on whether the structure it is copying lines up with its fixed grid. Two functions that differ only in knot placement get a perfect copy and a 7% error from the same KAN. That is the honest answer to a mathematician who works on free-knot splines: yes, the fixed grid is a real limitation, and here is exactly how it shows up.

For the stock-price results. Stock prices are not bent lines with known knots, and the paper uses smooth cubic splines, where Experiment B showed knot placement matters far less because no corner can be made anyway. So the example does not say the paper's KAN results are wrong. It says what kind of thing a KAN is: a sum of fixed-grid splines, exact when the data's structure happens to match the grid, approximate otherwise, and improving as the grid is refined at the cost of more parameters. That is a fair description to put in the paper.

Three honest limitations of the example itself. First, S2's knots being whole numbers means a grid of 10 catches them by chance (Table A, row 2); moving one knot to a non-integer value would make the point hold for every uniform grid. Second, the small residual errors on fine grids are a training-budget effect, not a representability one; more steps would remove them. Third, Experiment D used a simple search; better free-knot algorithms exist and that is Nadia's area, so she may want to replace it.

Reproducing everything. Run `python fit_kan.py` in the `orf_example` folder of the repository. It regenerates all numbers and figures in about three minutes. The data files are regenerated from Nadia's script and were checked against her originals to within 4×10^{-15}.

9. Glossary

Function. A rule that turns one number into another. Here, t in, S(t) out. Drawn as a line on a graph.

Piecewise linear. Made of straight segments joined end to end. A bent wire.

Knot / kink. A place where a piecewise-linear line changes direction. 'Knot' is the mathematical term, 'kink' the everyday one; in this document they mean the same thing.

Slope. How steeply a straight segment rises (positive) or falls (negative). S1 and S2 share the slopes 2, 1, 5, 2, 1 across their five pieces.

Hinge. The function max(0, t - c): flat until t reaches c, then rising in a straight line. Also called a ramp or hockey stick. Adding hinges builds any piecewise-linear line.

Spline. A curve stored as pieces joined at fixed points, with the heights (and in smooth splines, the curvature) chosen to match a target.

Grid. The fixed, evenly spaced positions at which a KAN's spline is allowed to bend. A 5-interval grid on 0 to 10 has join points at 0, 2, 4, 6, 8, 10. The model chooses heights at these points but cannot move the points.

Linear vs cubic spline (k = 1, k = 3). k = 1: straight pieces, can make sharp corners at grid points. k = 3: smooth curved pieces, no visible joins, cannot make a sharp corner anywhere.

Free-knot spline. A spline whose join points are themselves chosen to fit the data, rather than fixed in advance. More flexible, but finding the best positions is a difficult search.

KAN. Kolmogorov-Arnold Network. A learning model whose adjustable parts are curve shapes (splines) rather than plain numbers. Written by the number of units in each layer: [1,1] is one curve; [1,3,1] feeds the input through three curves whose outputs are summed and passed through one more.

Hidden layer. Units between the input and output. Curves feeding into curves. Lets the model form bends at positions not on any single grid.

Training. Searching for the model's settings (here, spline heights) that minimise the error against the target. Done by repeatedly nudging the settings in the direction that reduces error.

Step. One round of that nudging. We used 200.

L-BFGS. The particular search method used. Takes large, well-aimed steps; standard for small problems.

Worst error (max error). The largest gap between the model's output and the true value at any of the 1,001 points. The measure used in approximation theory.

RMSE. Root-mean-square error. An average-sized error, the measure used in the main paper. Less sensitive to a single bad point than the worst error.

Parameters / numbers learned. How many adjustable settings the model has. More parameters means more flexibility but a harder search.

Representability. Whether a model *could* copy the target exactly if its settings were perfect. S1 is representable by a 5-interval linear spline; S2 is not. Separate from whether training actually finds those settings.